

Python

- History and Overview
- Installing Python and Setting up IDEs (e.g., VS Code, PyCharm, Jupyter Notebook)
- Writing and Running Your First Python Program
- Python Syntax and Indentation
- Python Comments

Basic Python Concepts

1. Variables and Data Types

- Numbers (int, float, complex)
- Strings
- Booleans
- NoneType

2. Operators

- Arithmetic, Assignment, Comparison
- Logical, Bitwise, Membership, Identity

3. Input and Output

- Input from User
- Printing Outputs
- String Formatting (f-strings, format(), % operator)

4. Control Flow

- If-Else Statements
- Loops: For and While
- Break, Continue, Pass

Intermediate Python Concepts

1. Data Structures

- Lists: Creation, Methods, Comprehensions
- Tuples: Immutable Sequences
- Sets: Unique Elements and Operations
- Dictionaries: Key-Value Pairs, Comprehensions

2. Functions

- Defining Functions
- Arguments: Positional, Keyword, Default, Arbitrary
- Return Statement
- Lambda Functions
- Recursion
- Decorators

3. Modules and Packages

- Importing Built-in Modules
- Creating and Importing Custom Modules
- Using `pip` to Install External Libraries

4. File Handling

- Reading and Writing Text Files
- Working with CSV and JSON
- Exception Handling in File Operations

Advanced Python Concepts

1. Object-Oriented Programming (OOP)

- Classes and Objects
- Inheritance
- Polymorphism
- Encapsulation
- Magic/Dunder Methods (e.g., `__init__`, `__str__`, `__repr__`)

2. Error and Exception Handling

- Try, Except, Else, Finally
- Custom Exceptions

3. Iterators and Generators

- Creating and Using Iterators
- Using `yield` in Generators

4. Comprehensions

- Nested Comprehensions
- Set and Dictionary Comprehensions

Specialized Topics

1. Regular Expressions

- Pattern Matching
 - Search, Match, and Replace
- 2. **Multithreading and Multiprocessing**
 - Threading Module
 - Global Interpreter Lock (GIL)
 - Multiprocessing Module
- 3. **Networking**
 - Sockets
 - HTTP Requests with `requests` Library
- 4. **Working with Databases**
 - SQLite and `sqlite3` Module
 - CRUD Operations

Data Science and Visualization

1. **NumPy**: Arrays and Linear Algebra Operations
2. **Pandas**: DataFrames, Series, Data Cleaning
3. **Matplotlib**: Basic and Advanced Plotting
4. **Seaborn**: Statistical Visualization

Web Development

1. **Flask/Django**: Creating Web Applications
2. **APIs**: Building and Consuming REST APIs
3. **Web Scraping**: Using `BeautifulSoup` and `Scrapy`

Testing and Debugging

1. **Unit Testing**: Using `unittest`
2. **Debugging Tools**: `pdb` and IDE Debugging Features
3. **Code Profiling**: Using `cProfile`

Advanced Topics

1. **Machine Learning**
 - Using `scikit-learn`
 - Basic ML Models (Linear Regression, Classification)

2. Automation

- Automating Tasks with [selenium](#) and [pyautogui](#)

3. Packaging and Deployment

- Creating Python Packages
- Virtual Environments and Dependency Management

PyCharm

- **What is PyCharm?**

- PyCharm is an Integrated Development Environment (IDE) developed by JetBrains specifically for Python programming.
- It provides intelligent code assistance, debugging tools, and supports various libraries and frameworks.
- PyCharm comes in two editions: **Community Edition** (free) and **Professional Edition** (paid, with additional features like web development and databases).

- **Installing PyCharm**

- Download and Install from the official website: [PyCharm Download](#)
- Choose between Community and Professional Editions based on your requirements.

- **PyCharm Interface Overview**

- **Project Pane:** Displays files and folders in your project.
 - **Editor Window:** Where you write and edit your code.
 - **Tool Windows:** Integrated tools like Terminal, Version Control, Debugger, etc.
 - **Status Bar:** Shows the status of the current environment, such as the active Python interpreter, Git branch, etc.
 - **Navigation Bar:** Located at the top of the editor window, helping you navigate your project structure.
-

2. Setting Up PyCharm for Your Project

2.1 Creating a New Project

- Create a new Python project by selecting **File** → **New Project**.

- Choose the project type, typically **Pure Python**, and select the location to store your project.
- Optionally, create and set up a **virtual environment** (highly recommended for Python projects to avoid dependency conflicts).

2.2 Configuring the Python Interpreter

- **Setting up an interpreter** for the project (you can use a system-wide interpreter, virtual environment, or Docker container).
 - Go to **File** → **Settings** → **Project: <project_name>** → **Python Interpreter**.
 - You can add interpreters from local or remote sources and configure the environment based on your needs.

2.3 Working with Virtual Environments

- PyCharm can automatically create a virtual environment when creating a new project or you can create one manually.
 - Virtual environments ensure dependencies for each project are isolated from the system Python and other projects.
 - Use **pip** or **conda** (if using Anaconda) to manage dependencies within the virtual environment.
-

3. Code Writing and Editing

3.1 Code Completion and Intellisense

- PyCharm provides **auto-completion** and **code suggestions** to improve productivity.
 - You can press **Ctrl + Space** to trigger basic code completion.
 - PyCharm will offer method names, variable names, and possible completions based on context.

3.2 Refactoring Code

- PyCharm includes **refactoring** tools to help improve your code structure without changing its functionality.
 - Use **Refactor** from the right-click context menu or the **Shift + F6** shortcut to rename variables, functions, classes, and more.
 - **Extract Method/Variable**: Automatically extract repeated code into functions or variables for clarity.

3.3 Code Navigation

- **Go to Definition:** `Ctrl + B` to jump to the definition of a function or class.
- **Go to Symbol:** `Ctrl + Alt + Shift + N` to jump to any symbol, such as functions, variables, or classes, by name.
- **Find Usages:** `Alt + F7` shows where a particular class, function, or variable is used throughout the project.
- **Navigate Back/Forward:** `Ctrl + -` (back) and `Ctrl + Shift + -` (forward) to navigate through your code navigation history.

3.4 Multi-Cursor Editing

- Place multiple cursors in different places to edit them simultaneously:
 - Hold `Ctrl` (Windows/Linux) or `Cmd` (Mac) and click to place additional cursors.
 - Use **Find and Replace** with the `Ctrl + Shift + R` shortcut for mass changes.
-

4. Working with Files and Directories

4.1 File Operations

- **Create New Files and Directories:** Right-click in the project pane and select **New** to create Python files, directories, or other resources like HTML, JSON, etc.
- **Renaming and Moving Files:** Right-click and choose **Refactor** → **Rename** or drag and drop files within the Project pane.

4.2 Working with Multiple Tabs

- PyCharm supports **tabbed browsing** within the editor.
 - You can open multiple files simultaneously in tabs, and arrange them horizontally or vertically.
 - Use `Ctrl + E` (Windows/Linux) or `Cmd + E` (Mac) to switch between recently opened files.

4.3 File Comparisons

- **Compare Files:** PyCharm allows you to compare two versions of a file with the **Compare Files** option.
 - You can compare differences between local versions, or between different branches of a Git project.
-

5. Debugging with PyCharm

5.1 Setting Breakpoints

- Click on the left margin next to a line of code to set a breakpoint. A red dot appears.
- You can set breakpoints at key locations in your code to pause execution and inspect variables.

5.2 Running and Debugging Code

- **Run** the code with **Shift + F10** (Windows/Linux) or **Cmd + Shift + F10** (Mac).
- **Debug** with **Shift + F9** (Windows/Linux) or **Cmd + Shift + F9** (Mac).
 - The debugger pauses execution at breakpoints, allowing you to inspect variables, call stacks, and more.

5.3 Using the Debugger Console

- The Debugger Console lets you evaluate expressions, run code snippets, and interact with the current state of the program during debugging.

5.4 Conditional Breakpoints

- Right-click on a breakpoint to make it conditional, so it only pauses when a specific condition is met (e.g., when a variable equals a certain value).
-

6. Version Control (Git)

6.1 Initializing a Git Repository

- PyCharm integrates seamlessly with **Git**.
- Initialize a repository using **VCS** → **Enable Version Control Integration**, then select **Git**.

6.2 Git Operations

- **Commit Changes:** Use **Ctrl + K** (Windows/Linux) or **Cmd + K** (Mac) to commit changes to the repository.
- **Pull and Push:** Pull and push code using the VCS menu or Git toolbar.
- **Branching:** Create, switch, and merge branches using the VCS operations.

6.3 Viewing Git History

- PyCharm allows you to view file history, commit history, and even compare different commits or branches.
-

7. Testing in PyCharm

7.1 Running Tests

- PyCharm supports running tests directly from the IDE with frameworks like `unittest`, `pytest`, and `nose`.
- You can run a test file, test class, or individual test function by right-clicking on the test and selecting **Run**.

7.2 Writing Tests

- Use the **Generate** option (**Alt + Insert** on Windows/Linux or **Cmd + N** on Mac) to generate boilerplate test cases based on existing code.
- PyCharm integrates with `pytest` for more advanced testing scenarios, like fixtures, parametrization, and plugins.

7.3 Coverage Analysis

- PyCharm provides built-in tools to track test coverage with the **Run** → **Run with Coverage** option.
 - You can view which lines of code were executed during the tests.
-

8. PyCharm Tips and Tricks

8.1 Customizing PyCharm

- **Themes**: Choose from various themes like Dark, Light, or Custom.
- **Shortcuts**: Customize your keyboard shortcuts by navigating to **File** → **Settings** → **Keymap**.
- **Plugins**: Install useful plugins from **File** → **Settings** → **Plugins** (e.g., Markdown support, Database tools, Docker).

8.2 Code Formatting

- Use **Code** → **Reformat Code** (**Ctrl** + **Alt** + **L** on Windows/Linux or **Cmd** + **Option** + **L** on Mac) to format your code according to PEP8 standards.
- Customize code formatting settings by going to **File** → **Settings** → **Code Style**.

8.3 Using Live Templates

- Live Templates are predefined code snippets that can be expanded with a simple shortcut (e.g., **def** for function definitions).
- Define your own templates under **File** → **Settings** → **Live Templates**.

8.4 Searching in PyCharm

- **Find Files:** Use **Ctrl** + **Shift** + **N** (Windows/Linux) or **Cmd** + **Shift** + **O** (Mac) to search for a file.
- **Find in Files:** Use **Ctrl** + **Shift** + **F** (Windows/Linux) or **Cmd** + **Shift** + **F** (Mac) to search through the entire project for text or code.
- **Find Action:** Use **Ctrl** + **Shift** + **A** (Windows/Linux) or **Cmd** + **Shift** + **A** (Mac) to search for and execute any PyCharm action or command.